

BluePrint

Daniel Fernando García Bravo

Brayan Camilo Guerrero Guerrero

Nicolás Daniel Gutiérrez Suarez

Corporación Universitaria Minuto de Dios

Facultad de Ingeniería

Ingeniería de Sistemas

Bases de Datos Masivas – NRC 83362

Alonso Guevara Pérez

1. Resumen del proyecto

Este blueprint detalla el proceso de ingeniería para la migración de un ecosistema relacional de alta volumetría hacia una arquitectura de documentos orientada a escalabilidad. El sistema utiliza un motor de orquestación basado en microservicios para mitigar los riesgos de pérdida de datos y asegurar una transición fluida entre paradigmas de persistencia.

2. Especificaciones de la infraestructura

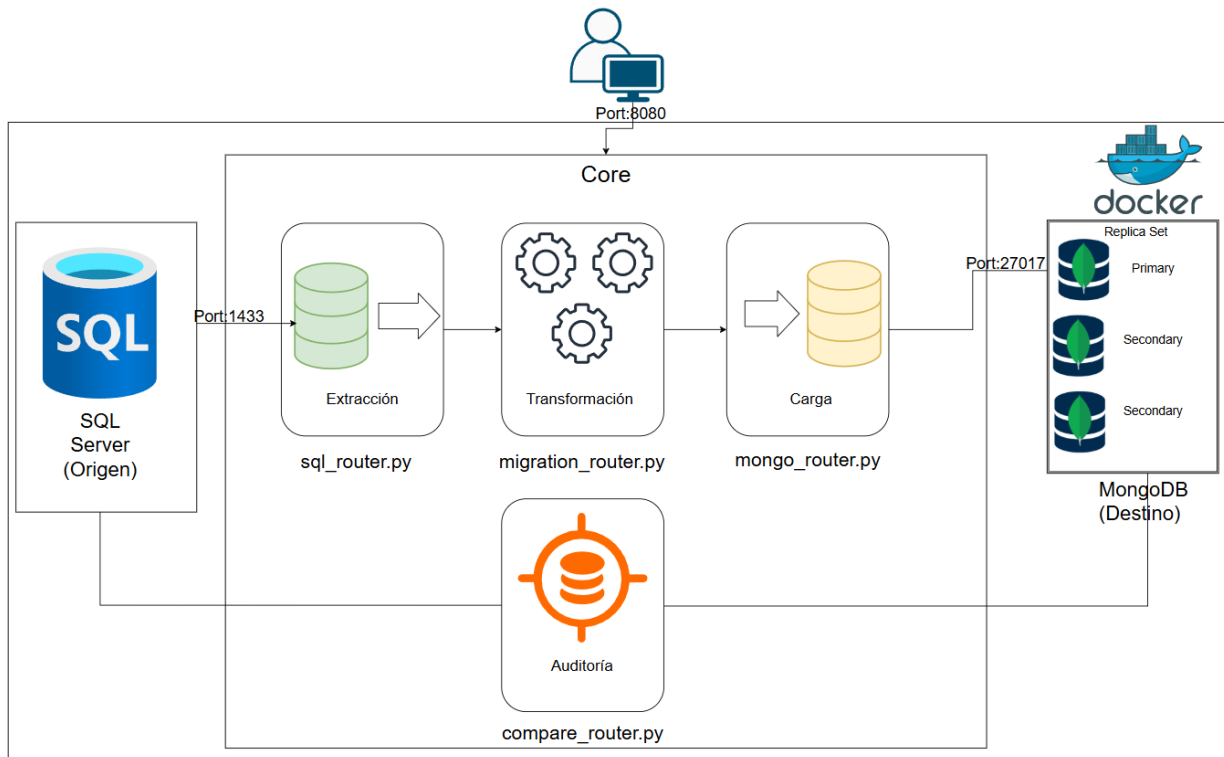
La infraestructura se basa en la contenedorización para garantizar la inmutabilidad del entorno de migración:

- Entorno de Ejecución: Orquestación mediante Docker Compose, lo que permite aislar los procesos de extracción y carga de la infraestructura física del host.
- Gestión de Secretos: Uso de archivos .env para la inyección de variables de entorno, evitando la exposición de credenciales críticas de SQL Server y MongoDB en el código fuente.
- Aprovisionamiento Automático: Los contenedores de base de datos se inicializan mediante scripts de configuración (setup.sh) y restauración de esquemas (restore-db.sql) para asegurar que el punto de partida sea consistente en cada ejecución.

3. Diseño de la arquitectura de datos

La migración no es un espejo, es una reestructuración de la información:

- **Origen (Relacional):** Datos estructurados en tablas con integridad referencial rígida en SQL Server.
- **Destino (Documental):** Datos desnormalizados en colecciones de MongoDB, optimizados para lecturas de baja latencia mediante el uso de objetos BSON.
- **Lógica de Transformación:** Implementada en la capa de aplicación (migration_router.py), donde se resuelven las relaciones de tipo *One-to-Many* y *Many-to-Many* para convertirlas en documentos embebidos o referencias documentales según el caso de uso.



4. PIPELINE ETL: FLUJO DE EJECUCIÓN MODULAR

El proceso de migración se divide en tres fases críticas orquestadas por enrutadores especializados:

A. Fase de Extracción (sql_router.py)

- **Lectura Eficiente:** Implementación de consultas SQL optimizadas para extraer datos en bloques (chunks) para no saturar el buffer de red ni la memoria RAM del contenedor.
- **Sanitización Inicial:** Limpieza de tipos de datos incompatibles directamente en la fuente.

B. Fase de Procesamiento (migration_router.py)

- **Mapping de Esquemas:** Traducción de las filas del RDBMS a diccionarios de Python listos para ser convertidos a JSON.
- **Gestión de Identidades:** Generación de identificadores únicos (ObjectIDs) manteniendo la trazabilidad con las llaves primarias originales para auditorías posteriores.

C. Fase de Ingesta (mongo_router.py)

- **Bulk Operations:** Uso de inserciones por lotes para maximizar el rendimiento del motor de MongoDB.
- **Validación de Escritura:** Confirmación de inserción para asegurar que el pipeline no continúe si hay una falla de conexión en el clúster.

5. Monitoreo y Control de Calidad

Para asegurar el éxito de la operación, se integran herramientas de observabilidad y validación:

- **Dashboard de Control:** Interfaz web (dashboard.html) conectada a la aplicación principal para el seguimiento visual del progreso de la migración.
- **Motor de Auditoría (compare_router.py):** Ejecución de scripts de contraste post-migración que comparan el *Checksum* y el conteo de registros entre SQL Server y MongoDB para certificar que no hubo pérdida de información durante el tránsito.

6. Gestión de Riesgos y Plan de Contingencia (RollBack)

- **Protocolo de Fallo:** Si se detecta una degradación de rendimiento mayor al 20% en los contenedores o errores de integridad, el sistema pausa automáticamente la carga.
- **Puntos de Recuperación:** Al basarse en procesos de solo lectura en la fuente (SQL Server), el riesgo de corrupción de los datos originales es nulo.
- **Recuperación Rápida:** En caso de error crítico, se procede al truncamiento de las colecciones de MongoDB afectadas y se reinicia el pipeline desde el último bloque validado exitosamente.